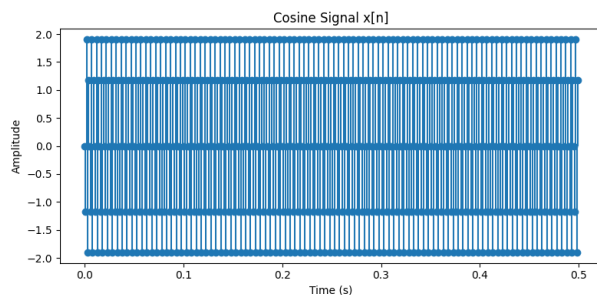


Module 2 Lab 2 Report

학번 비공개 심현성

[Task 1]

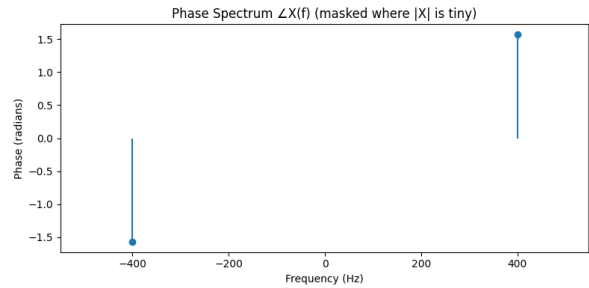
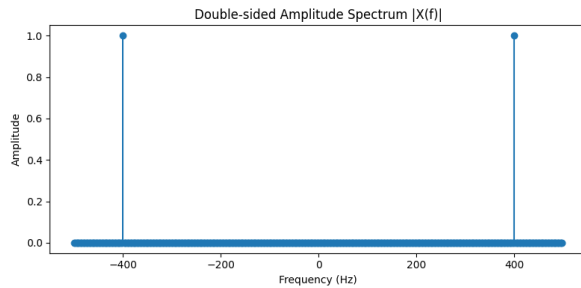
1. $x(t) = A \cos(2\pi ft + \theta_0)$ 를 이용하여 cosine signal을 생성하였다. 하단 그래프는 우측 코드의 실행 결과인 sampled cosine signal이다.



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # ----- Parameters -----
5 A = 2.0          # Cosine amplitude
6 f0 = 400.0       # Cosine frequency in Hz
7 theta0 = np.pi / 2 # Initial phase in radians
8 fs = 1000.0      # Sampling rate in Hz
9 L = 0.5          # Signal duration in seconds
10
11 N = int(L * fs)   # Number of samples
12 t = np.arange(N) / fs
13
14 # ----- Signal -----
15 x = A * np.cos(2*np.pi*f0*t + theta0) # pure cosine
16
17 # ----- Plot -----
18 # Time-domain reference
19 plt.figure(figsize=(8,4))
20 plt.stem(t, x, basefmt=" ")
21 plt.title("Cosine Signal x[n]")
22 plt.xlabel("Time (s)")
23 plt.ylabel("Amplitude")
24 plt.tight_layout()
25 plt.show()
```

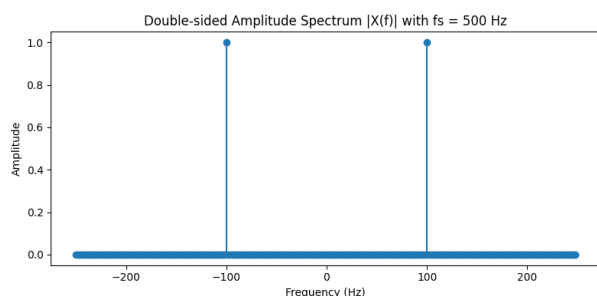
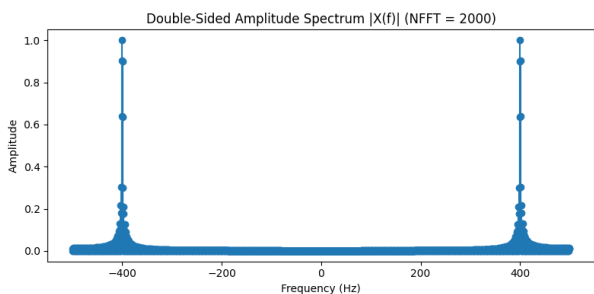
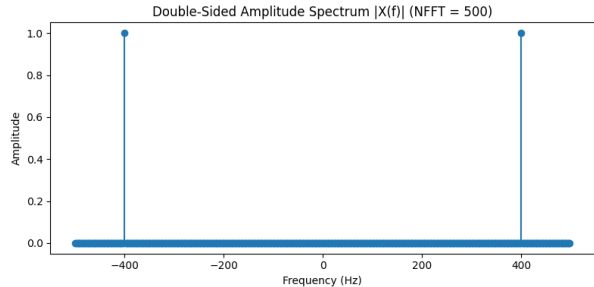
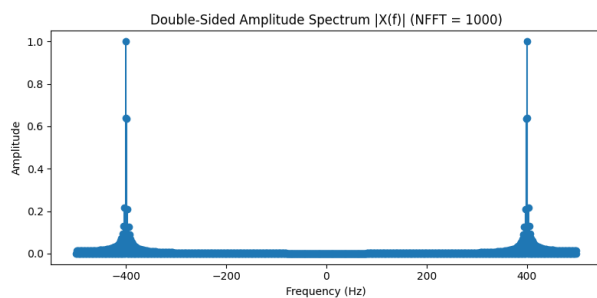
```
1 # ----- FFT (double-sided) -----
2 Y = np.fft.fft(x)
3 freqs = np.fft.fftfreq(N, 1/fs) # frequency axis (double-sided)
4
5 # Magnitude
6 amp = np.abs(Y) / N
7
8 # Phase (mask bins with ~zero amplitude)
9 phase = np.angle(Y)
10 eps = 1e-6
11 phase_masked = np.where(amp > eps, phase, np.nan)
12
13 # ----- Plots (one figure per chart) -----
14 # Magnitude spectrum
15 plt.figure(figsize=(8,4))
16 plt.stem(freqs, amp, basefmt=" ")
17 plt.title("Double-sided Amplitude Spectrum |X(f)|")
18 plt.xlabel("Frequency (Hz)")
19 plt.ylabel("Amplitude")
20 plt.tight_layout()
21 plt.show()
22
23 # Phase spectrum
24 plt.figure(figsize=(8,4))
25 plt.stem(freqs, phase_masked, basefmt=" ")
26 plt.title("Phase Spectrum ∠X(f) (masked where |X| is tiny)")
27 plt.xlabel("Frequency (Hz)")
28 plt.ylabel("Phase (radians)")
29 plt.tight_layout()
30 plt.show()
```

2. Magnitude spectrum에서는 ± 400 Hz에서 약 1의 peak가 나타났다. Phase spectrum의 경우, 의미 있는 위상 정보는 ± 400 Hz에서만 나타나며 각각 $+\frac{\pi}{2}$ 와 $-\frac{\pi}{2}$ 의 값을 갖는다. 작은 magnitude를 갖는 bin의 위상은 masking 처리하였다. 하단의 두 그래프는 각각 좌측 코드의 실행 결과인 magnitude vs frequency와 phase vs frequency 그래프이다.



3. NFFT를 500, 1000, 2000으로 변화시키며 스펙트럼의 변화를 관찰하였다. 하단의 세 그래프를 보면, NFFT가 커질수록 frequency bin spacing이 감소하므로 주파수축이 더 촘촘해지고, 그 결과 스펙트럼의 peak가 보다 부드럽고 세밀하게 표현된다. 다만 이는 원래 신호에 새로운 정보가 추가된 것이 아니라, zero-padding을 통해 스펙트럼을 더 세밀한 주파수축에서 관찰한 결과이다. 세 경우 모두 ± 400 Hz에서 주요 peak가 나타나며, 원래 입력한 cosine 신호의 주파수 성분이 유지됨을 확인할 수 있다.

```
1 # ----- FFT with different NFFT -----
2 NFFT_list = [500, 1000, 2000]
3
4 for NFFT in NFFT_list:
5     Y = np.fft.fft(x, n=NFFT)
6     freqs = np.fft.fftfreq(NFFT, 1/fs)
7     amp = np.abs(Y) / len(x)
8
9     plt.figure(figsize=(8,4))
10    plt.stem(freqs, amp, basefmt=" ")
11    plt.title(f"Double-Sided Amplitude Spectrum |X(f)| (NFFT = {NFFT})")
12    plt.xlabel("Frequency (Hz)")
13    plt.ylabel("Amplitude")
14    plt.tight_layout()
15    plt.show()
```



4. 좌측 그래프는 sampling rate를 $f_s = 500$ Hz로 감소시킨 후 FFT를 수행한 결과이다. 이 경우 Nyquist frequency는 $f_N = \frac{f_s}{2} = 250$ Hz가 되며, 원래 신호의 주파수인 400 Hz는 Nyquist frequency를 초과하므로

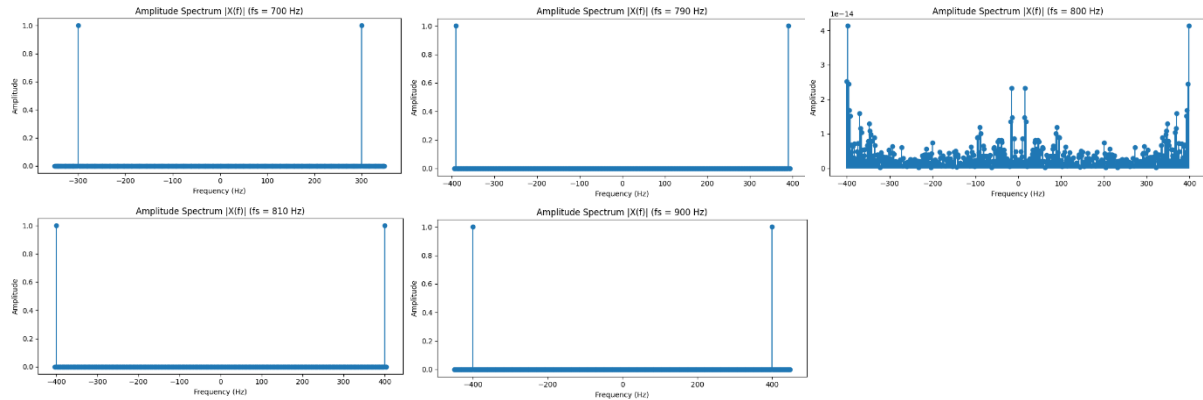
로 aliasing이 발생한다. 그 결과, 스펙트럼에서는 기존 [Task 1-2]의 그래프와 같은 ± 400 Hz가 아니라 ± 100 Hz에서 peak가 나타나게 된다. 이는 aliasing 주파수 $f_{alias} = |f_0 - f_s| = 100$ Hz에 해당하며, 샘플링 주파수가 충분히 크지 않을 경우 원래 신호의 주파수가 왜곡되어 나타남을 확인할 수 있다.

```
1 # ----- Parameters -----
2 fs = 500.0          # Reduced sampling rate (Hz)
3
4 N = int(L * fs)     # Number of samples (N = 250)
5 t = np.arange(N) / fs
6
7 # ----- Signal -----
8 x_alias = A * np.cos(2*np.pi*f0*t + theta0)
9
10 # ----- FFT -----
11 Y = np.fft.fft(x_alias, n=N)
12 freqs = np.fft.fftfreq(N, 1/fs)
13 amp = np.abs(Y) / N
14
15 # ----- Plot -----
16 plt.figure(figsize=(8,4))
17 plt.stem(freqs, amp, basefmt=" ")
18 plt.title("Double-sided Amplitude Spectrum |X(f)| with fs = 500 Hz")
19 plt.xlabel("Frequency (Hz)")
20 plt.ylabel("Amplitude")
21 plt.tight_layout()
22 plt.show()
```

```
1 # ----- Parameters -----
2 fs_list = [700, 790, 800, 810, 900]
3
4 # ----- Test different sampling rates -----
5 for fs in fs_list:
6     N = round(L * fs)
7     t = np.arange(N) / fs
8     x = A * np.cos(2*np.pi*f0*t + theta0)
9
10     Y = np.fft.fft(x, n=N)
11     freqs = np.fft.fftfreq(N, 1/fs)
12     amp = np.abs(Y) / N
13
14     # Use only non-negative frequencies for identifying the peak
15     pos_mask = freqs >= 0
16     freqs_pos = freqs[pos_mask]
17     amp_pos = amp[pos_mask]
18
19     peak_idx = np.argmax(amp_pos)
20     peak_freq = freqs_pos[peak_idx]
21
22     print(f"fs = {fs:4.0f} Hz, peak frequency =  $\pm$  {peak_freq:.1f} Hz")
23
24     plt.figure(figsize=(8,4))
25     plt.stem(freqs, amp, basefmt=" ")
26     plt.title(f"Amplitude Spectrum |X(f)| (fs = {fs} Hz)")
27     plt.xlabel("Frequency (Hz)")
28     plt.ylabel("Amplitude")
29     plt.tight_layout()
30     plt.show()
```

5. Sampling frequency를 여러 값(700, 790, 800, 810, 900 Hz)으로 변화시키며 magnitude spectrum에서 최대 amplitude가 나타나는 주파수를 확인하였다. 실험 결과, $f_s < 800$ Hz에서는 Nyquist 조건을 만족하지 못하여 aliasing이 발생하였고, 이에 따라 최대 amplitude가 나타나는 주파수가 400 Hz가 아닌 더 낮은 주파수로 이동하는 것을 확인할 수 있었다. 반면, $f_s > 800$ Hz에서는 aliasing이 발생하지 않아 ± 400 Hz에서 peak가 관찰되었다. 한편, $f_s = 800$ Hz에서는 원래 신호 주파수 400 Hz가

Nyquist frequency와 정확히 일치하지만, 초기 위상이 $\frac{\pi}{2}$ 이므로 샘플링된 신호가 이론적으로 모두 0이 되어 의미있는 peak가 정상적으로 나타나지 않았다. 따라서 원래 신호의 주파수 성분을 안정적으로 관측하기 위해서는 $f_s > 800$ Hz여야 한다. 하단의 그래프와 표는 여러 f_s 에 따른 FFT 결과를 나타낸 것이다.



f_s [Hz]	700	790	800	810	900
peak frequency [Hz]	± 300	± 390	± 398	± 400	± 400

[Task 2]

1. I2S_recording.wav 파일의 signal length는 240000 samples, sampling rate는 48000 Hz이다.

```

1 import soundfile as sf
2
3 # ----- Audio Read -----
4 x, fs = sf.read("I2S_recording.wav")
5
6 # ----- Check signal information -----
7 N = len(x)
8
9 print(f"Sampling rate: {fs} Hz")
10 print(f"Signal length: {N} samples")
11 print(f"Signal duration: {N/fs:.4f} s")

```

Sampling rate: 48000 Hz

Signal length: 240000 samples

Signal duration: 5.0000 s

```

1 # ----- Sensitivity Scaling -----
2 sensitivity_db = -23.0
3 sensitivity_linear = 10**(sensitivity_db / 20)
4
5 x_pa = x / sensitivity_linear
6
7 print(f"Sensitivity: {sensitivity_db} dB/Pa")
8 print(f"Linear sensitivity: {sensitivity_linear:.4f}")
9 print(f"Maximum pressure: {np.max(np.abs(x_pa)):.4f} Pa")
10 print("Signal was scaled to Pa.")

```

Sensitivity: -23.0 dB/Pa

Linear sensitivity: 0.0708

Maximum pressure: 1.5201 Pa

Signal was scaled to Pa.

2. 센서의 sensitivity가 -23dB/Pa 로 주어졌으므로, 이를 선형 스케일로 변환하면 $S = 10^{-\frac{23}{20}} \approx 0.0708$ 이다. 이를 이용하여 recorded signal을 sensitivity의 선형값으로 나누어 Pa 단위로 스케일링하였다.

3. Pa scale로 변환한 신호에 대해 rfft()와 rfftfreq()를 사용하여 single-sided linear magnitude spectrum을 계산하고 시각화하였다.

```
1 # ----- FFT (single-sided) -----
2 N = len(x_pa)
3 Y = np.fft.rfft(x_pa)
4 freqs = np.fft.rfftfreq(N, 1/fs)
5
6 # ----- Single-sided magnitude scaling -----
7 amp = np.abs(Y) / N
8 if N % 2 == 0: # even N -> Nyquist frequency exists
9     amp[1:-1] *= 2.0
10 else: # odd N -> no Nyquist bin
11     amp[1:] *= 2.0
12
13 # ----- Plot -----
14 plt.figure(figsize=(8,4))
15 plt.plot(freqs, amp)
16 plt.title("Single-Sided Linear Magnitude Spectrum")
17 plt.xlabel("Frequency (Hz)")
18 plt.ylabel("Magnitude (Pa)")
19 plt.tight_layout()
20 plt.show()
```

